

Flattening the Curve

Implementing the SQL on FHIR ViewDefinition in .NET

Brendan Kowitz, Microsoft

HL7 FHIR DevDays

The largest FHIR-only
event in the world

Minneapolis, MN
www.devdays.com



HL7 FHIR DevDays 2026 | Minneapolis, MN | June 15-18, 2026 | @HL7 | @FirelyTeam | #fhirdevdays | www.devdays.com

ORGANIZED BY

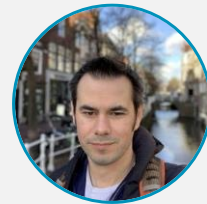


HL7®, FHIR® and the flame Design mark are the registered trademarks of Health Level Seven International and are used with permission.

1

Who am I?

- **Brendan Kowitz**
- EM for FHIR OSS, Azure Health Data Services (FHIR)
- Originally from Australia; lived in the Pacific Northwest for the past 10+ years
- github.com/brendankowitz



GitHub

github.com/brendankowitz/FHIRDevDays2026

2

2

SQL on FHIR libraries exist. Just not in .NET.

THE ECOSYSTEM TODAY

	JavaScript	HL7 reference implementation	
	Python	Community libraries	
	Java	Production tooling	
	.NET		

3

3

BEFORE THIS PROJECT

You're on .NET and want SQL on FHIR.

A few options...

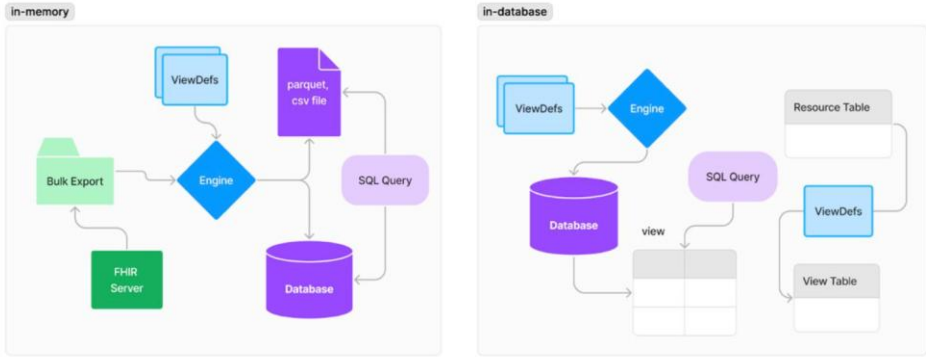
 Run the reference impl out-of-process Shell out to the JS/Python runner — now you're running a non-.NET runtime inside a .NET stack.	 Build the ViewDefinition layer yourself You own the flattening and every spec-conformance edge case — and re-own it each release.	 Use database view architecture Translate ViewDefinitions to SQL inside a FHIR-native database. Powerful — but a full architectural commitment that may not fit your .NET stack.
---	--	--

4

4

There are two popular categories of runners:

- **In-memory runners** consume resources, flatten, and output results into a stream, a file or a table. You can imagine the ETL pipeline from FHIR Bulk export ndjson files transformed into parquet files.
- **In-database runners** translate ViewDefinition into SQL query over an FHIR-native database. In that case the view can be a real database view or table. In-database runners could be far more efficient than in-memory in speed and storage resources but are much more complex for implementers.



<https://build.fhir.org/ig/FHIR/sql-on-fhir-v2/>

Ignixa.SqlOnFhir

INTRODUCING



Native .NET 9/10

No JVM. No Python. No sidecar.



Multi-version

STU3 · R4 · R4B · R5 · R6-ballot - one codebase.



MIT licensed

Open source. Contributions welcome.



Conformance-tested

Passes the official SQL on FHIR v2 test suite.

```
get started

# library
dotnet add package Ignixa.SqlOnFhir

# cli (global tool)
dotnet tool install -g \
  Ignixa.SqlOnFhir.Cli

# run your first view
ignixa-sqlonfhir r4 preview \
  --views patient-view.json
```

What is “Ignixa”

Ignixa started as a personal open-source project for rethinking FHIR/.NET from a clean slate.



Research

A proving ground for new FHIR/.NET ideas, built to learn and explore. Tackling even advanced parts of the spec.



Architecture

Rethinks FHIR on modern .NET: Ground up Parsing, Specification, Serialization, FHIRPath with clean, modular packages.

Publishing 20+ nuget packages. FHIRPath Engine has 45K downloads

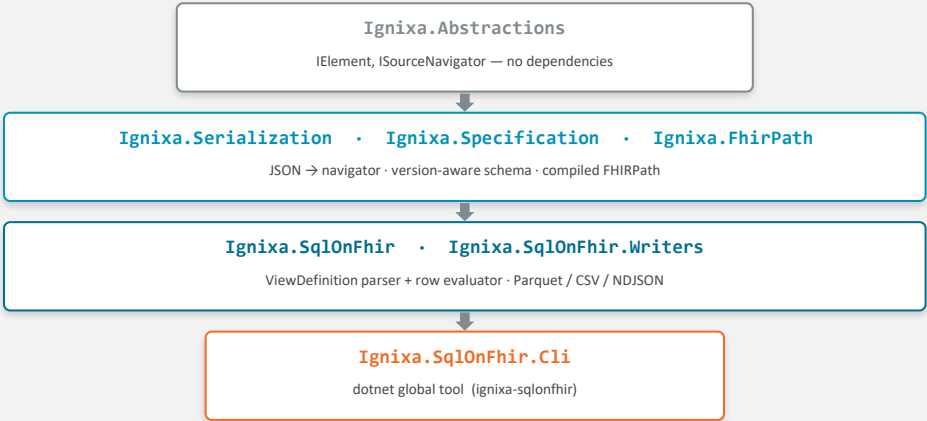


Built with frontier AI

8+ years of my own experience, distilled into code with the latest AI coding tools.

Modular. Pick what you need.

PACKAGE ANATOMY



Fast enough for \$export pipelines

FHIRPath performance

Not a wrapper — a ground-up, visitor-pattern engine with optimization passes at parse time and fast paths at run time.



Compile time

- **Constant folding** `1 + 2 → 3`
- **Short-circuit** `false and f() → skip`
- **Algebraic simplify** `x.where(true) → x`
- **Static type-checking** `bad path → error`



Run time

- **Cached delegates** `2nd eval skips compile`
- **Allocation-free paths**
- **Interpreter fallback**

One AST, many visitors — the optimizer, analyzer, and delegate compiler are all just visitors.

Parse → Evaluate → Write

THREE STAGES. CLEAN SEPARATION.



1 · Parse

`ViewDefinitionExpressionParser`
`.Parse()`

Compiles all FHIRPath upfront. Fail fast - invalid FHIRPath is caught here, not at evaluation time.



2 · Evaluate

`SqlOnFhirEvaluator`
`.Evaluate(view, resource)`

WHERE filters · SELECT extracts · forEach unnests arrays into rows · %resource reaches the root. Pure & stateless.



3 · Write

`ParquetFileWriter`
`Csv · Ndjson · Create your own`

Consumes the row stream.

A ViewDefinition example

THE SHAPE OF THE THING

```
patient-view.json

{
  "resourceType": "ViewDefinition",
  "resource": "Patient",
  "where": [{ "path": "active = true" }],
  "select": [{ "column": [
    { "name": "id", "path": "id", "type": "string" },
    { "name": "family_name", "path": "name.where(use='official').first().family" },
    { "name": "given_name", "path": "name.where(use='official').first().given.first()" },
    { "name": "birth_date", "path": "birthDate", "type": "date" },
    { "name": "gender", "path": "gender", "type": "string" }
  ] } ]
}
```

11

11

One resource. Many rows.

FOREACH EXAMPLE

```
name-rows.json

{
  "resource": "Patient",
  "select": [
    { "column": [
      { "name": "id", "path": "id" } ] },
    { "forEach": "name", "column": [
      { "name": "name_use", "path": "use" },
      { "name": "family", "path": "family" },
      { "name": "given_name", "path": "given.first()" }
    ] }
  ]
}
```

Result — one row per name

id	name_use	family	given
pt-001	official	Smith	Alice
pt-001	maiden	Jones	Alice

The first select group emits **id** once.
The **forEach** group repeats per element in the **name** array — standard SQL unnesting, expressed as a ViewDefinition.
Use **%resource** inside a **forEach** to reference the root resource.

12

12

No subprocess. No tables. <10 lines.

FROM C#

```

Program.cs

var schema = FhirVersion.R4.GetSchemaProvider();

var viewDef = JsonSourceNodeFactory.Parse(
    File.ReadAllText("patient-view.json")).ToSourceNavigator();

var patient = JsonSourceNodeFactory.Parse(
    File.ReadAllText("patient.json")).ToElement(schema);

var evaluator = new SqlOnFhirEvaluator();
foreach (var row in evaluator.Evaluate(viewDef, patient))
    Console.WriteLine($"{row["id"]} - {row["family_name"]}, {row["birth_date"]}");

```

13

13

Works with Firely SDK too.

FROM THE FIRELY SDK

```

Program.cs

using Ignixa.Extensions.FirelySdk;

var schema = FhirVersion.R4.GetSchemaProvider();
var viewDef = JsonSourceNodeFactory.Parse(
    File.ReadAllText("patient-view.json")).ToSourceNavigator();

// Parse with the Firely SDK, then bridge into Ignixa
ISourceNode node = FhirJsonNode.Parse(File.ReadAllText("patient.json"));
var patient = node.ToElement(schema);

var evaluator = new SqlOnFhirEvaluator();
foreach (var row in evaluator.Evaluate(viewDef, patient))
    Console.WriteLine($"{row["id"]} - {row["family_name"]}, {row["birth_date"]}");

```

14

14

Let's run it.

DEMO · CLI

 preview → validate → run → batch

```
# data pre-staged in demo/data/ (NDJSON patients +  
# a patient ViewDefinition, generated by Ignixa.FhirFakes)  
  
$ ignixa-sqlonfhir r4 preview --views patient-view.json
```

The test data is generated with Ignixa.FhirFakes — full story in “Fluent, Fast, and Fake” — tomorrow (Jun 17), 9:15 AM · Ski-U-Mah

CLI quick reference

KEEP THIS UP DURING THE DEMO

Command	What it does
ignixa-sqlonfhir r4 preview --views <file>	Show schema without data
... preview --views <f> --input <f> --rows 10	Show sample rows
ignixa-sqlonfhir r4 validate --views <dir>	Validate all ViewDefinitions
... run --views <dir> --input <dir> --out d/ --format csv	Batch CSV output
--var name=value	Override ViewDefinition constants at runtime

Pattern: <version> <command> [options] Versions: [stu3](#) [r4](#) [r4b](#) [r5](#) [r6](#)

What this unlocks for .NET

\$EXPORT → VIEWDEFINITIONS → PARQUET → DATA LAKE



BI dashboards

Power BI / Fabric on Parquet directly — no ETL middleware



CI validation

Validate ViewDefinitions against schema before deployment



Research cohorts

Reproducible ViewDefinitions as the extraction contract



Multi-version

Same ViewDefinition, different FHIR version flag



Data lake pipelines

\$export NDISON → ViewDefinitions → Parquet, one command



RELATED TALK - WED, JUNE 17

Real-Time SQL Views: Subscriptions × SQLonFHIR

Jared Erwin · 3:45 PM · Johnson Great

Try it.

THREE COMMANDS TO GET STARTED

```
terminal

# library
dotnet add package Ignixa.SqlOnFhir

# cli
dotnet tool install -g Ignixa.SqlOnFhir.Cli

# run your first ViewDefinition
ignixa-sqlonfhir r4 preview \
  --views patient-view.json
```



GitHub

github.com/brendankowitz/ignixa-fhir



Docs

brendankowitz.github.io/ignixa-fhir



MIT licensed

Contact

You can find / reach me here:

-  **Speaker's Gallery**
Via the DevDays app
-  **Email**
bkowitz@microsoft.com
-  **Bluesky**
[@brendankowitz.bsky.social](https://bsky.app/profile/brendankowitz.bsky.social)
-  **GitHub**
github.com/brendankowitz/FHIRDevDays2026

Q&A

-  **GitHub**
github.com/brendankowitz/FHIRDevDays2026

Feedback

- Enjoyed this talk? I'd love your feedback — it genuinely helps.
- Scan the QR code, or visit:
- www.devdays.com/feedback

